

物联网安全技术实验报告 3-4

班级	物联网 2202	学号	223428010210
专业	物联网工程	姓名	陈梓欣
实验表现成绩		实验成绩	
报告成绩			

一、实验题目：Blundo 传感器密钥分配方案、哈希函数

二、实验地点：410-3

三、实验目的：

- 1 理解流密码算法原理，编程实现 A5/1, RC4 等加密算法；
- 2 理解 AES 原理，编程实现列混合运算过程；
- 3 编程并实现 Blundo 传感器密钥分配方案；
- 4 安装配置 cryptodome 库并实现分组加密 DES，AES 加密算法；
- 5 应用 cryptodome 库函数，实现 RSA 加密算法，签名算法；

四、实验内容

（完成题目代码及运行结果并截图保存，在注释中含学号、姓名信息）

1. A5/1 流密码算法

223428010210 陈序斌

```
#A5/1 流密码加密
plaintext_str = input("输入加密的二进制字符串 {0,1}*:")
# 初始化 X Y Z 寄存器
X_lst=[1,0,1,0,1,0,1,0,1,0,1,0,1,0,1,0,1]
Y_lst=[1,1,0,0,1,1,0,0,1,1,0,0,1,1,0,0,1]
Z_lst=[1,1,1,0,0,0,0,1,1,1,1,0,0,0,1,1,1,0,0,0,1]
plaintext_lst=[]
ciphertext_lst=[]
key_lst=[]
if len(plaintext_str)>0:
    #Verify the correct of input
    for i in plaintext_str:
        if int(i) in (0,1):
            plaintext_lst.append(int(i))
        else:
            print("输入错误")

    if len(plaintext_lst)>0:
        count_1 =0
        count_0 =0
        maj =0
        # Maj Function x8 y10 z10多数决得到maj
        for i in plaintext_lst:

            if X_lst[8] == 1:
                count_1=count_1+1
            else:
                count_0=count_0+1
            if Y_lst[10] == 1:
                count_1=count_1+1
            else:
                count_0=count_0+1
            if Z_lst[10] == 1:
                count_1=count_1+1
            else:
                count_0=count_0+1
            if count_1 > count_0 :
                maj =1
            else:
                maj=0
        #根据 Maj 决定 X Y Z 更新
        if X_lst[8]==maj:
            print("X steps")
            t = (X_lst[13]+X_lst[16]+X_lst[17]+X_lst[18])%2
            X_lst.pop(-1)
            X_lst.insert(0,t)
        if Y_lst[10]==maj:
            print("Y steps")
            t = (Y_lst[20]+Y_lst[21])%2
            Y_lst.pop(-1)
            Y_lst.insert(0,t)
        if Z_lst[10]==maj:
            print("Z steps")
            t = (Z_lst[7]+Z_lst[20]+Z_lst[21]+Z_lst[22])%2
            Z_lst.pop(-1)
            Z_lst.insert(0,t)
        #3个寄存器工作一轮后, 每个寄存器输出最后位置的信息生成key
        key_bit = (X_lst[-1]+Y_lst[-1]+Z_lst[-1])%2
        encrypt_bit = (i+key_bit)%2
        key_lst.append([key_bit])
        ciphertext_lst.append(encrypt_bit)
        print("消息流为: ",plaintext_str)
        print("密钥流为: ",key_lst)
```

```
输入加密的二进制字符串 {0,1}*: 01100
X steps
Z steps
消息流为: 01100
密钥流为: [[1]]
X steps
Y steps
消息流为: 01100
密钥流为: [[1], [1]]
X steps
Y steps
Z steps
消息流为: 01100
密钥流为: [[1], [1], [0]]
Y steps
Z steps
消息流为: 01100
密钥流为: [[1], [1], [0], [0]]
Z steps
消息流为: 01100
密钥流为: [[1], [1], [0], [0], [1]]
```

2. RC4 流密码算法

```
# 223428010210 陈梓欣

#RC4 流密码加密
#RC4的Python实现--若初·知乎

#初始化S
# def ini_S(R):
#     S=[x for x in range(256)]
#     j=0
#     for i in range(256):
#         j = (j+S[i]+S[j]) % 256
#         S[i],S[j]=S[j],S[i]

#     return S
def ini_S(R):
    S = [x for x in range(256)]
    j = 0
    for i in range(256):
        j = (j + S[i] + R[i]) % 256
        S[i], S[j] = S[j], S[i]
    return S

#根据key长度初始化辅助表R
def gen_R(key):#生成辅助表R
    temp=bytes(key,encoding='utf-8')
    temp=list(temp)
    print('private key:',temp)
    len_key=len(temp)
    R=[temp[i%len_key] for i in range(256)]

    return R

#流密码生成过程
def stream_k(S,length):
    i=0
    j=0
    Sh=[]
    length=int(length)
    for i in range(length):
        i=(i+1)%256
        j=(j+S[i])%256
        S[i],S[j]=S[j],S[i]
        h=(S[j]+S[i])%256
        k=S[h]
        Sh.append(k)
    return Sh

choose=input('choose 1 for encryption, 2 for decryption:')
if choose=='1':
    key=input('input the key:')
    R=gen_R(key)
    S=ini_S(R)
    print('S:',S)
    plaintext=input('input the plaintext:')
    ciphertext=''
    Sh=stream_k(S,len(plaintext))
    for i in range(len(plaintext)):
        ciphertext = ciphertext + '%02x'%(Sh[i] ^ ord(plaintext[i]))
    print('ciphertext:',ciphertext)
if choose=='2':
    key=input('input the key:')
    R=gen_R(key)
    S=ini_S(R)
    ciphertext=input('input the ciphertext:')
    plaintext=''
    Sh=stream_k(S,len(ciphertext)/2)
    for i in range(int(len(ciphertext)/2)):
        plaintext = plaintext + chr(int(ciphertext[0:2,16] ^ Sh[i]))
        ciphertext=ciphertext[2:]
    print('plaintext:',plaintext)

choose 1 for encryption, 2 for decryption: 2
input the key: test
private key: [116, 101, 115, 116]
input the ciphertext: c6ea4b7d8c455d52b93c3107
plaintext: hello world
```

3. Blundo 传感器密钥分配方案原理

```
#223428010210 陈梓欣

#Blundo 传感器密钥分配方案
#服务器端
from random import randint
# 随机生成 199~999 范围内的一个素数作为模数 p
def randprime():
    top = randint(199, 999)
    for x in range(top, 200, -1):
        for y in range(2, x):
            if x % y == 0:
                break
        else:
            return x
# 中心节点：服务器端
class Server:
    def __init__(self):
        self.a = randint(10, 99)
        self.b = randint(10, 99)
        self.c = randint(10, 99)
        self.p = randprime()
        self.chd_index = 0 # 子节点编号计数器
        self.IDs = {} # 记录所有子节点的 ID

    def __str__(self):
        return f"对称多项式为:  $f(x,y) = \{self.a\} + \{self.b\}*(x+y) + \{self.c\}*x*y \bmod \{self.p\}$ "
    # 生成多项式的描述
    def poly_generate(self):
        return f"生成多项式为:  $\{self.a\} + \{self.b\}*(x+y) + \{self.c\}*x*y \bmod \{self.p\}$ "
    # 为一个新子节点生成 ID 和一次函数  $A + Bx \bmod p$ 
    def get_ID(self):
        self.chd_index += 1
        chd_name = 'c' + str(self.chd_index)
        ID = randint(10, 199)
        self.IDs[chd_name] = ID
        A = (self.a + self.b * ID) % self.p
        B = (self.b + self.c * ID) % self.p
        return [chd_name, ID, A, B, self.p]

    def get_p(self):
        return self.p
# 子节点类：客户端
class Client:
    def __init__(self, name, id, A, B, p):
        self.name = name
        self.id = id
        self.A = A
        self.B = B
        self.p = p
        self.recvIDs = {} # 其他节点 ID 表
        self.shareKeys = {} # 与其他节点的共享密钥表

    def getID(self):
        return self.id

    def getName(self):
        return self.name

    def __str__(self):
        return f"{self.name}: ID={self.id}, A={self.A}, B={self.B}, p={self.p}"
# 交换两个节点的 ID
def swapKey(c1, c2):
    c1.recvIDs[c2.getName()] = c2.getID()
    c2.recvIDs[c1.getName()] = c1.getID()
# 使用共享一次函数和对方 ID 生成共享密钥
def shareKey(c1, c2):
    id2 = c1.recvIDs[c2.getName()]
    id1 = c2.recvIDs[c1.getName()]
    key1 = (c1.A + c1.B * id2) % c1.p
    key2 = (c2.A + c2.B * id1) % c2.p
    assert key1 == key2, f"密钥不一致! {c1.getName()}={key1}, {c2.getName()}={key2}"
    c1.shareKeys[c2.getName()] = key1
    c2.shareKeys[c1.getName()] = key2
```

```

# 方案测试部分
if __name__ == "__main__":
    # 初始化中心服务器
    s = Server()
    print(s)
    print(s.poly_generate())
    # 创建3个子节点
    [name1, id1, A1, B1, p] = s.get_ID()
    c1 = Client(name1, id1, A1, B1, p)
    print(c1)

    [name2, id2, A2, B2, p] = s.get_ID()
    c2 = Client(name2, id2, A2, B2, p)
    print(c2)

    [name3, id3, A3, B3, p] = s.get_ID()
    c3 = Client(name3, id3, A3, B3, p)
    print(c3)
    # 节点交换 ID
    swapKey(c1, c2)
    swapKey(c1, c3)
    swapKey(c2, c3)
    # 节点生成共享密钥
    shareKey(c1, c2)
    shareKey(c1, c3)
    shareKey(c2, c3)
    # 输出共享密钥
    print("\nc1 与其他节点共享密钥: ", c1.shareKeys)
    print("c2 与其他节点共享密钥: ", c2.shareKeys)
    print("c3 与其他节点共享密钥: ", c3.shareKeys)

```

对称多项式为: $f(x,y) = 62 + 91*(x+y) + 62*x*y \bmod 977$

生成多项式为: $62 + 91*(x+y) + 62*x*y \bmod 977$

c1: ID=156, A=580, B=970, p=977

c2: ID=92, A=618, B=910, p=977

c3: ID=126, A=781, B=87, p=977

c1 与其他节点共享密钥: {'c2': 913, 'c3': 675}

c2 与其他节点共享密钥: {'c1': 913, 'c3': 969}

c3 与其他节点共享密钥: {'c1': 675, 'c2': 969}

4. AES 列混合运算原理和算法实现

```
# 223428010210 陈梓欣

# 有限域GF(2^8)乘法定义
def xtime(a):
    return ((a << 1) ^ 0x1B) & 0xFF if a & 0x80 else (a << 1)

def gf_mul(a, b):
    result = 0
    for i in range(8):
        if b & 1:
            result ^= a
            a = xtime(a)
            b >>= 1
    return result

# 列混合 (对一列进行)
def mix_single_column(col):
    t = col[0] ^ col[1] ^ col[2] ^ col[3]
    u = col[0]
    col[0] ^= t ^ gf_mul(col[0] ^ col[1], 0x02)
    col[1] ^= t ^ gf_mul(col[1] ^ col[2], 0x02)
    col[2] ^= t ^ gf_mul(col[2] ^ col[3], 0x02)
    col[3] ^= t ^ gf_mul(col[3] ^ u, 0x02)
    return col

# 对整个状态 (4x4矩阵) 做列混合
def mix_columns(state):
    for i in range(4):
        column = [state[row][i] for row in range(4)]
        mixed = mix_single_column(column)
        for row in range(4):
            state[row][i] = mixed[row]
    return state

# 示例状态矩阵 (16字节明文)
state = [
    [0x87, 0xF2, 0x4D, 0x97],
    [0x6E, 0x4C, 0x90, 0xEC],
    [0x46, 0xE7, 0x4A, 0xC3],
    [0xA6, 0x8C, 0xD8, 0x95]
]

print("原始状态矩阵: ")
for row in state:
    print([hex(x) for x in row])

mixed_state = mix_columns(state)

print("\n列混合后的状态矩阵: ")
for row in mixed_state:
    print([hex(x) for x in row])
```

原始状态矩阵:

```
['0x87', '0xf2', '0x4d', '0x97']
['0x6e', '0x4c', '0x90', '0xec']
['0x46', '0xe7', '0x4a', '0xc3']
['0xa6', '0x8c', '0xd8', '0x95']
```

列混合后的状态矩阵:

```
['0x47', '0x40', '0xa3', '0x4c']
['0x37', '0xd4', '0x70', '0x9f']
['0x94', '0xe4', '0x3a', '0x42']
['0xed', '0xa5', '0xa6', '0xbc']
```

5. 分组加密算法 DES，加密解密过程

```
# 223428010210 陈梓欣

#DES加密解密
from Crypto.Cipher import DES
from binascii import b2a_hex, a2b_hex

class MyDESCrypt: #自己实现的DES加密类
    def __init__(self, key = ''):
        #密钥长度必须为64位，也就是8个字节
        if key != '':
            self.key = key.encode('utf-8')
        else:
            self.key = '12345678'.encode('utf-8')
        self.mode = DES.MODE_CBC
        # 加密函数，如果text不足16位就用空格补足为16位，
        # 如果大于16当时不是16的倍数，那就补足为16的倍数。
    def encrypt(self, text):
        try:
            text = text.encode('utf-8')
            cryptor = DES.new(self.key, self.mode, self.key)
            # 这里密钥key 长度必须为16 (DES-128)，
            # 24 (DES-192)，或者32 (DES-256) Bytes 长度
            # 目前DES-128 足够目前使用
            length = 16 #Lenth可以设置为8的倍数
            count = len(text)
            if count < length:
                add = (length - count)
                # \0 backspace
                # text = text + ('\0' * add)
                text = text + ('\0' * add).encode('utf-8')
            elif count > length:
                add = (length - (count % length))
                # text = text + ('\0' * add)
                text = text + ('\0' * add).encode('utf-8')
            self.ciphertext = cryptor.encrypt(text)
            # 因为DES加密时候得到的字符串不一定是ascii字符集的，输出到终端或者保存时候可能存在问题
            # 所以这里统一把加密后的字符串转化为16进制字符串
            return b2a_hex(self.ciphertext)
        except:
            return ""
        # 解密后，去掉补足的的空格用strip() 去掉
    def decrypt(self, text):
        try:
            cryptor = DES.new(self.key, self.mode, self.key)
            plain_text = cryptor.decrypt(a2b_hex(text))
            # return plain_text.rstrip('\0')
            return bytes.decode(plain_text).rstrip('\0')
        except:
            return ""

msg = "password is 961223"
key = "12345678" #key值可传可不传
des1 = MyDESCrypt()
#加密
cipherTxt = des1.encrypt(msg) #返回值为bytes型
print(cipherTxt)
#解密
decTxt = des1.decrypt(cipherTxt); #返回值为str型
print(decTxt)

b'159cf72c0bd2a818da3e332a358aae118df99257ad0ba75ab40840a240a13f34'
password is 961223
```

6. 分组加密算法 AES 加密解密过程

```
# 223428010210 陈梓欣

from os import urandom
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad

# 原始消息
message = b'We study in sau1'
print('加密前消息为: ', message)

# 生成16字节密钥和16字节IV
secret_key = urandom(16)
iv = urandom(16)

print('加密密钥为: ', secret_key)
print('初始化向量为: ', iv)

# 创建AES加密器对象 (CBC模式)
cipher_encrypt = AES.new(secret_key, AES.MODE_CBC, iv)

# 使用PKCS7填充消息 (填充到16的倍数)
padded_message = pad(message, AES.block_size)

# 执行加密
encrypted_text = cipher_encrypt.encrypt(padded_message)
print('加密后密文为: ', encrypted_text)

# === 解密过程 ===
# 创建AES解密器对象, 使用相同的key和IV
cipher_decrypt = AES.new(secret_key, AES.MODE_CBC, iv)

# 解密密文
decrypted_padded = cipher_decrypt.decrypt(encrypted_text)

# 去除填充, 得到原始消息
decrypted_message = unpad(decrypted_padded, AES.block_size)
print('解密后原始消息为: ', decrypted_message)
```

加密前消息为: b'We study in sau1'
加密密钥为: b'\xdae\x1dz\x9d\xd4\xff\xc5\x8a\xcd\x16\xa1L\xe6\$'
初始化向量为: b'LK&\x9e\xe6\xe4d(L\xb1\xdbp\x1bs\xcd\xfd'
加密后密文为: b'_\x88%9\xddT\xcc6\xdd\xb5\xaaav\xa6a\x04z\xc0\x96s\xfc\x95o\x8e}\xfc\xef?\xf4M5\xd9\xb5'
解密后原始消息为: b'We study in sau1'

7. 公钥加密算法 RSA 加密解密过程

```
# 223428010210 陈梓欣

#数字签名
from Crypto import Random
from Crypto.Hash import SHA
from Crypto.Cipher import PKCS1_v1_5 as Cipher_pkcs1_v1_5
from Crypto.Signature import PKCS1_v1_5 as Signature_pkcs1_v1_5

from Crypto.PublicKey import RSA
import base64

# 加密解密：公钥加密，私钥解密
# 签名验证：私钥签名，公钥验证
print ("1、生成私钥和公钥，运行在服务器上，TTP")
# 伪随机数生成器
random_generator = Random.new().read
# rsa算法生成实例
rsa = RSA.generate(1024, random_generator)
# A的私钥对的生成并保存
private_pem = rsa.exportKey()
with open('aaa-private.pem', 'wb') as f:
    f.write(private_pem)
# A的公钥对的生成并保存
public_pem = rsa.publickey().exportKey()
with open('aaa-public.pem', 'wb') as f:
    f.write(public_pem)
# B的私钥对的生成并保存
private_pem = rsa.exportKey()
with open('bbb-private.pem', 'wb') as f:
    f.write(private_pem)
# B的公钥对的生成并保存
public_pem = rsa.publickey().exportKey()
with open('bbb-public.pem', 'wb') as f:
    f.write(public_pem)
# 加密和解密
print ("2、加密和解密")
#A发送消息给B，用对方的公钥加密消息
# A使用B的公钥对内容进行rsa 加密
message = '大家好，这就是我要加密的数据'
print ("message: " + message)
#读取文件获得B的公钥
with open('bbb-public.pem') as f:
    key = f.read()
rsakey = RSA.importKey(str(key))
cipher = Cipher_pkcs1_v1_5.new(rsakey)
cipher_text = base64.b64encode(cipher.encrypt(bytes(message.encode("utf8"))))
print ("加密(encrypt)")
print (cipher_text)
# B使用自己的私钥对内容进行rsa 解密
with open('bbb-private.pem') as f:
    key = f.read()
rsakey = RSA.importKey(key)
cipher = Cipher_pkcs1_v1_5.new(rsakey)
text = cipher.decrypt(base64.b64decode(cipher_text), random_generator)
print ("解密(decrypt)")
print ("text:" + str(text,"utf8"))
print("message:"+message)
assert str(text,"utf8") == message

1、生成私钥和公钥，运行在服务器上，TTP
2、加密和解密
message: 大家好，这就是我要加密的数据
加密(encrypt)
b'hkh1/Dkzgy9gpMbQeF9NGxU2kQm/KDtgt7BGd7TYexajXf50TjFhr0B0NLfB89x17ZXlHdCpgwemK8kA7Sf+MzXA/4BeAv7CTI/8Xp9RgD4m0QkD3HdHjEpC/ZkCdTc3fmnNuf1dmtdl96hnsUT
e0QmGmGDY/8/TGuE9SA1NKtM='
解密(decrypt)
text:大家好，这就是我要加密的数据
message:大家好，这就是我要加密的数据
```

8. 公钥加密算法 RSA 签名过程

```
# 223428010210 陈神欣

from Crypto import Random
from Crypto.Hash import SHA
from Crypto.Signature import PKCS1_v1_5 as Signature_pkcs1_v1_5
from Crypto.Cipher import PKCS1_v1_5 as Cipher_pkcs1_v1_5
from Crypto.PublicKey import RSA
import base64

random_generator = Random.new().read

# 1. 生成发送方密钥对（签名用）
signer_rsa = RSA.generate(1024, random_generator)
with open('sender-private.pem', 'wb') as f:
    f.write(signer_rsa.exportKey())
with open('sender-public.pem', 'wb') as f:
    f.write(signer_rsa.publickey().exportKey())

# 2. 生成接收方密钥对（加密用）
receiver_rsa = RSA.generate(1024, random_generator)
with open('receiver-private.pem', 'wb') as f:
    f.write(receiver_rsa.exportKey())
with open('receiver-public.pem', 'wb') as f:
    f.write(receiver_rsa.publickey().exportKey())

print("密钥对生成完毕\n")

# 发送方操作
def sender_process(message: str):
    print("发送方消息:", message)

    # 读取发送方私钥，签名消息
    with open('sender-private.pem') as f:
        sender_private_key = RSA.importKey(f.read())
    signer = Signature_pkcs1_v1_5.new(sender_private_key)

    # 计算消息摘要
    digest = SHA.new(message.encode('utf-8'))

    # 生成签名
    signature = signer.sign(digest)
    signature_b64 = base64.b64encode(signature).decode('utf-8')
    print("生成签名(Base64):", signature_b64)

    # 读取接收方公钥，进行消息加密
    with open('receiver-public.pem') as f:
        receiver_public_key = RSA.importKey(f.read())
    cipher = Cipher_pkcs1_v1_5.new(receiver_public_key)

    encrypted_message = cipher.encrypt(message.encode('utf-8'))
    encrypted_message_b64 = base64.b64encode(encrypted_message).decode('utf-8')
    print("加密后的消息(Base64):", encrypted_message_b64)

    return encrypted_message_b64, signature_b64

# 接收方操作
def receiver_process(encrypted_message_b64: str, signature_b64: str):
    # 读取接收方私钥，解密消息
    with open('receiver-private.pem') as f:
        receiver_private_key = RSA.importKey(f.read())
    cipher = Cipher_pkcs1_v1_5.new(receiver_private_key)

    encrypted_message = base64.b64decode(encrypted_message_b64)
    sentinel = Random.new().read(15) # 解密失败时返回随机字节
    decrypted_message = cipher.decrypt(encrypted_message, sentinel).decode('utf-8')
    print("\n接收方解密消息:", decrypted_message)

    # 读取发送方公钥，验证签名
    with open('sender-public.pem') as f:
        sender_public_key = RSA.importKey(f.read())
    verifier = Signature_pkcs1_v1_5.new(sender_public_key)

    digest = SHA.new(decrypted_message.encode('utf-8'))
    signature = base64.b64decode(signature_b64)

    if verifier.verify(digest, signature):
        print("✅ 签名验证成功，消息可信")
    else:
        print("❌ 签名验证失败，消息可能被篡改")

# 测试流程
if __name__ == "__main__":
    original_message = "RSA签名加密完整流程测试"
    encrypted_msg, signature = sender_process(original_message)
    receiver_process(encrypted_msg, signature)
```

密钥对生成完毕

发送方消息: RSA签名加密完整流程测试

生成签名(Base64): OQKPwuJi30Ss4Ci8xCsytoF1x7cG8Bq50qFkK1e7zg1FQXpm2lHm/cqA0cnIrbmdWxpJ3VyEFv+1nZ08Djzja1rpgTXJzVRFi3REBwK37DY28R3q24wxfwLreeI765FaopP8IuWzxHuprteQHn9sSMC3rmnx4S8Ng3D0YKwttPQ=

加密后的消息(Base64): iW858QH54GkI/tjj8LobMr0pxXvIur5DPKM5uXbDGzLYKD1SqIqeQzSdpMtnUuBbR7F/R5CsTww+xywP2Q5ZpFRR+z47IbA6yESR0c0ce7F2b9L0rdnINvTrxIt1l8xaIe1zmneZw8vz2cWkC9WCOAQ1TeGkKkiu1CuHjA6Y7Y=

接收方解密消息: RSA签名加密完整流程测试

✅ 签名验证成功，消息可信

五、实验中遇到的问题及解决方法

问题：RC4 流密码算法 `ini_S` 中 KSA 错误：没有使用 `R[i]`，导致密钥无效！

解决方法：修改为： $j = (j + S[i] + R[i]) \% 256$

六、实验总结及心得

通过本次实验，我深入理解了现代密码学的核心算法及其应用场景。首先，在流密码算法如 RC4 和 A5/1 的实现中，体会到密钥流生成对加密强度的重要性。其次，通过对 AES 算法的学习，尤其是列混合运算的编程实现，加深了对有限域运算的理解，感受到了 AES 强大的安全性保障。Blundo 传感器密钥分配方案让我认识到分布式密钥管理的复杂性及多项式秘密共享的巧妙设计。通过 Cryptodome 库实现 DES、AES 和 RSA 算法，掌握了对称加密和非对称加密的编程实践，特别是 RSA 签名与验签过程，强化了数字签名保障数据完整性与身份验证的认识。本实验不仅提升了我的编程能力，更加深了对密码学理论与实际应用结合的理解，为未来的信息安全学习和研究奠定了坚实基础。